サイバーセキュリティ 3 レジュメ 2

2026 年 10 月 26 日

- 講義はオンデマンドもあり
- 成績は期末レポートのみ

目次

1 直近のセキュリティ動向 2
1.1 RANSOMWARE BRIEF：ここ 1 ヶ月の国内ランサムウェア事件（2026.09.25 —10.25） . . 3
1.2 実例：AI を武器化したサイバー攻撃ツール 6
1.3 ネットワーク防御要素 (PEP) 9
1.4 ネットワーク防御要素 (PEP) 10
1.5 ネットワーク防御要素 (PEP) 10
1.6 サイバー攻撃の手法 (1)：マルウェア 12
1.7 ちょっと休憩：あなたの情報漏れてませんか？ 15
1.8 サイバー攻撃の手法 (2)：ネットワーク攻撃と認証 15
1.9 データベースの概要 17
1.10 SQL インジェクション 20

1　直近のセキュリティ動向

2026 年 7-10 月に発生した情報漏えい事案と AI 活用攻撃ツールの実態

1.0.1　企業からの情報漏えい事案（2026 年 7-10 月）

企業からの情報漏えいが相次いでいます。主要事例（100 万件以上）を時系列で整理すると、攻撃手法は主に「不正アクセス」「ランサムウェア」「マルウェア」に分類されます。

- **7 月 31 日 EPARK リラク & エステ:** 不正アクセス / 約 3,300 万件（可能性）
- **6 月 23 日 KDDI 株式会社:** 不正アクセス / 最大 1,422 万件（可能性）
- **10 月 5 日焼肉きんぐ（物語コーポレーション）:** 不正アクセス / 1,078 万件（可能性）
- **8 月 1 日 E ストアー（ショップサーブ）:** 不正アクセス / 885 万件（可能性）
- **10 月 8 日第一興商（カラオケ）:** マルウェア感染 / 約 872 万件（確定）

まとめ

2026 年 7 月から 10 月にかけて、民間企業において数百万から数千万件規模の大規模な個人情報漏えい事案が多発しました。主な原因は「不正アクセス」や「マルウェア感染」であり、顧客情報を大量に保有するサービス業（美容、通信、飲食、EC、カラオケなど）が主な標的となっています。

例え話による解説

これは例えるなら、頑丈な金庫を持っているはずの銀行（企業）に、泥棒（攻撃者）が裏口の鍵穴からこっそり侵入（不正アクセス）したり、清掃員に変装して毒ガスを撒き散らしたり（マルウェア感染）して、大量の顧客リストをごっそり盗み出したような状況です。一度侵入の隙を見つけられると、被害はあっという間に何千万という規模に膨れ上がってしまいます。

1.0.2　大学・行政機関からの情報漏えい事案（2026 年 7-10 月）

大学・行政機関からの情報漏えいも確認されています。ここでは公的機関特有のリスク（委託先経由、サポート詐欺への脆弱性）が浮き彫りになっています。

まとめ

公的機関や教育機関でも情報漏えいが発生しています。特徴として、自組織への直接的な攻撃だけでなく、「業務の外部委託先を経由した漏えい」や、職員が騙されてしまう「サポート詐欺による被害」など、サプライチェーンの弱点や人的要因（ヒューマンエラー）を突かれるリスクが高いことが分かります。

例え話による解説

例えるなら、立派なお城（公的機関）の正門は固く閉ざして守っていても、出入りしている商人（委託先）が道中で襲撃されて通行証を奪われたり、門番が「お城の修理屋です」と嘘をつく詐欺師（サポート詐欺）をあっさり信用して裏口を開けてしまったりするようなものです。いくらお城の壁を高くしても、関わる人や業者の隙を突かれれば情報は簡単に漏れてしまいます。

1.1 RANSOMWARE BRIEF：ここ 1 ヶ月の国内ランサムウェア事件（2026.09.25 ―10.25）

公表・報道ベースのまとめ｜講義・勉強会資料

1.1.1 この 1 ヶ月を表す数字

※2026 年 10 月 25 日時点の公表情報に基づく

- **495 組織:** クラウド 1 事業者への攻撃で影響を受けた企業・自治体（IDC フロンティア）
- **約 500 台:** 大阪公立大学で全停止したサーバー
- **53 社:** 中核メンバーが拘束された Qilin による国内被害（警察庁）

まとめ

この 1 ヶ月間のランサムウェア被害の深刻さを示す「495 組織」「約 500 台」「53 社」という数字は、ひとつの拠点やシステムが攻撃されることで、いかに広範囲に被害が連鎖するか（サプライチェーンの弱点）を物語っています。

例え話による解説

例えるなら、ショッピングモールの「電気室（クラウドや共通基盤）」がたった一人の不審者に占拠されただけで、モールに入っている「495 のテナント（企業・自治体）」すべてが営業停止に追い込まれたような状態です。自分のお店は戸締まりをしっかりしていても、大元のインフラがやられるとどうにもならなくなってしまう怖さがあります。

1.1.2 タイムライン：1 ヶ月で大学・鉄道・クラウド・SIer が相次いで被害

出典：各組織の公式発表、警察庁（2026.10.08）、piyolog、TT Incidents Log ほか

- **09.25 佐賀大学:** 事務系 NAS が暗号化
- **09.26 京王グループ:** 決済・予約・ポイントが停止（鉄道運行は影響なし）
- **09.30 池上通信機:** 外部の通報で発覚・暗号化確認
- **10.02 大阪公立大学:** 約 500 台停止・10/8 まで全授業休講
- **10.07 IDC フロンティア:** 495 の企業・自治体に影響
- **10.08 警察庁発表:** Qilin 中核とみられる人物を拘束しドイツへ引き渡し
- **10.13 銀のさら等:** 委託先サーバ被害・最大約 650 万件
- **10.16 Qilin が犯行を主張:** 京王の内部資料とされる画像をリークサイトに掲載（未確認）
- **10.20 NEC:** 再委託先経由で顧客環境が暗号化

まとめ

わずか 1 ヶ月の間に、教育機関、鉄道インフラ、クラウド事業者、食品デリバリーなど、あらゆる業種でランサムウェア被害が連続して発生しました。特に「外部からの通報で発覚」や「委託先・再委託先を経由した被害」が目立っており、自社の監視だけでは防ぎきれない現状が浮き彫りになっています。

例え話による解説

まるで伝染病が街のあちこちで同時に発生しているような状況です。しかも、学校の生徒から感染したのではなく、「給食を運んでくる業者（委託先）」からウイルスが持ち込まれたり、病気になったことに自分では気づけず「お隣さん（外部の通報）」に言われて初めて発覚したりと、見えないルートからの感染ルートが脅威となっています。

1.1.3 CASE 01: IDC フロンティア（IDCF クラウド） | 2026.10.07 発生

1 つのクラウド基盤が止まり、495 組織に波及

東日本リージョン 1 の 4 ゾーンで仮想サーバーが停止・再起動不可に。県・区の Web サイトや J リーグクラブのアプリなど、直接攻撃されていない組織のサービスが止まりました。

発表内容の推移:

- **10.07:** 原因はランサムウェア攻撃。該当リージョンを遮断・停止し、他リージョンの管理コンソールも停止
- **10.08:** 該当ゾーンの顧客データは取り出し・復元が困難。復元は顧客側のバックアップのみとの見通し
- **10.14:** 外部へのデータ流出はなかったと判断。スナップショット利用の 194 社はデータ取り出しが可能に。侵入経路は調査中

※ **注意:** 攻撃者は「ハイパーバイザー 239 台」「7 分で暗号化」などと画像で主張しましたが、IDC フロンティアはこれらの数値を一切認めていません（未検証の主張）。

まとめ

クラウド基盤という「共通インフラ」が標的になったことで、直接攻撃を受けていない多数の組織（県や区、スポーツクラブなど）のシステムが同時にダウンしました。「クラウド事業者のバックアップも復元困難」となるケースがあるため、利用者側で独自のバックアップ（責任共有モデル）を持つことの重要性が改めて示された事例です。

例え話による解説

これは、みんなが大切な荷物を預けている「巨大な貸し倉庫」に強盗が入り、倉庫の入り口の鍵を勝手に変えられてしまったような事件です。「私は毎月家賃を払って預けているのに !」と言っても、倉庫側も扉を開けられません。大事なものは、家（自社）にもコピー（バックアップ）を置いておかなければならないという教訓です。

1.1.4　CASE 02・03: 入口は小さくても、止まる範囲は大きい

大阪公立大学 × NEC 再委託先（再委託先の「正規アカウント」で侵入）:

- 再委託先（9 月下旬に被害）　NEC 顧客環境（認証情報を悪用）　大学の基盤（約 500 台停止）
- 仮想化基盤が暗号化・改ざんされ、バックアップの多くも暗号化で使用不能
- 10/8 まで全授業休講、10/9 に対面授業を再開
- 13 万件超の個人情報流出のおそれ（10/5 会見時点）
- 原因判明（10/20-21）まで約 2 週間

京王電鉄グループ（被害サーバー 2 台、影響はグループ全体）:

- 被害サーバーはグループのサーバーと京王プラザホテルのサーバーの 2 台（報道）
- ホテル予約、ストアのキャッシュレス決済、百貨店のポイントなどが停止

まとめ

攻撃者は「正規の ID やパスワード」を悪用して侵入するため、セキュリティシステムに異常として検知されにくくなっています。また、侵入されたのは末端の「再委託先」や「たった 2 台のサーバー」であっても、そこから内部ネットワークを辿られて共通基盤全体が暗号化され、大学の授業休講やグループ全体の決済停止といった大規模な業務停止を引き起こします。

例え話による解説

例えるなら、下請けの清掃業者が持っている「合い鍵（正規アカウント）」が盗まれたようなものです。泥棒は窓ガラスを割る（サイバー攻撃）のではなく、合い鍵を使って堂々と正面玄関から入ってくるため、警報機が鳴りません。そしてビルの中で「マスターキー」を見つけ出し、たった一部屋の侵入から、ビル全館のドアをロックして使えなくしてしまうのです。

1.1.5　主要ランサムウェアグループの動向

摘発されても亜種で復活——イタチごっこの構造が続いています。

- **LockBit 5.0 (復活):** 2024 年 2 月「Operation Cronos」で摘発されるも、2025 年 9 月に「LockBit 5.0」で復活
- **BlackCat / ALPHV / Noberus (活動停止):** 2021 年 11 月〜/ 世界初 Rust 言語製 RaaS。報酬 80-90%、Change Healthcare から $22M 身代金。2023 年 12 月米司法省撲滅作戦、2024 年 2 月に活動停止（Exit Scam の疑い）
- **Clop (Cl0p) (大規模脅威):** ロシア系 / 2019 年〜/ 大規模データ窃取型。GoAnywhere、Accellion などゼロデイ脆弱性を悪用。2023 年「MOVEit」脆弱性攻撃で 2,700 組織以上に甚大な被害をもたらす
- **BlackSuit / Royal (重要インフラ標的):** Conti 系後継 / 2022 年〜/ 二重脅迫型。医療・教育・政府機関など米国重要インフラを標的。高度なハイブリッド暗号化を使用。CISA や FBI から共同で強い警告対象

まとめ

世界のランサムウェアグループは、国際的な警察の作戦で摘発されても、名前を変えたり新しいバージョン（LockBit 5.0 など）を作ってすぐに復活する「イタチごっこ」の状況にあります。最近は特定の組織ではなく、多くの企業が使うソフトウェアの弱点（ゼロデイ脆弱性）を狙ったり、医療や教育などの重要インフラを専門に狙うなど、手口がより凶悪かつ大規模化しています。

例え話による解説

これはまるで、国際指名手配の凶悪な犯罪シンジケートです。警察がアジトを突入して幹部を捕まえても（摘発）、残党が別の国で看板を掛け替えて（名前を変えて）すぐに同じ悪事を始めます。しかも最近は、一件ずつ家を狙うのではなく、街の水道管に毒を入れるような方法（共通ソフトの脆弱性狙い）で、一度に何千人からもお金を巻き上げようとする手口に変わってきています。

1.2 実例：AI を武器化したサイバー攻撃ツール

2026 年 10 月、韓国金融機関を襲った AI 攻撃ツールの実態

1.2.1 攻撃ツール例

- **ARTEX (高リスク):**
  - 中国製 AI ペネトレーションテストエージェント
  - 標的指定　脆弱性探索　43 秒で侵入　データ流出
  - 被害者数: 68,000 件 / 対象機関: 9 行 / 侵入時間: 43 秒 / 確認状況: 確定
  - 新韓銀行: 25,000 件、KB 国民銀行: 19,000 件
  - 出典: JBpress, Reuters, CrowdStrike
- **Bissa Scanner:**
  - React2Shell 脆弱性を利用した攻撃自動化ツール
  - スキャン　CVE-2025-55182　認証情報収集　S3 アップロード
  - 侵害企業: 900+ 社 / .env ファイル: 30,000+ / 活動期間: 2026 年 4 月 / 対象国: グローバル / 認証情報: 65,000+ / ZIP アーカイブ: 400+
  - 出典: The DFIR Report, SocPrime
- **AI Enabled Workflow:**
  - 攻撃者 (@BonJoviGoesHard | Dr. Tube) が Claude Code、Telegram (@bissa_scan_bot | BissaPwned)、OpenLaw、Bissa Scanner などを連携させ、CVE-2025-55182 への Exploit、認証情報奪取、S3 バケット (filebase) への転送を自動化

まとめ

AI はもはや人間を支援するだけでなく、サイバー攻撃を「完全自動化・高速化」する強力な武器として実戦投入されています。代表例の「ARTEX」は、AI が自分で弱点を探し、わずか数十秒でシステムに侵入・データを盗み出すなど、人間の手作業では不可能なスピードと規模で被害をもたらしています。

例え話による解説

例えるなら、これまでの泥棒が「家を一つずつ下見して、鍵のかかっていない窓を探す」という地道な作業をしていたのに対し、AI 攻撃ツールは「ドローンの大群を街中に放ち、数秒で何万軒もの家の鍵穴の形をスキャンし、一瞬でピッキングして中のものを持ち去る」ようなものです。人間が「怪しいドローンがいる」と気づいてから警察を呼ぼうとしても、すでに手遅れになってしまいます。

1.2.2 ARTEX による実被害——韓国から日本、そして世界へ

ARTEX は実際のサイバー攻撃に使われ、被害は拡大中です。

- **韓国・金融機関 (2026 年 9-10 月):** 流出個人情報約 100,000 件。新韓銀行で 2 万 5000 人分流出 (9/30)。金融機関 16 社へ被害拡大。融資申請等の認証回避。犯人は中国在住 26 歳推定
- **日本企業への攻撃 (2026 年 10-11 月):** 標的組織約 80 社。金融業種を中心に被害集中。GMO サイバー犯罪対策特定。脆弱性自律探索と情報抜き取り。警察組織・国の事業にも波及
- **世界規模の攻撃 (2026 年 9 月以降):** 標的組織 2,250 組織以上。中国政府系「Storm-0940」悪用。世界中の組織を広範に標的。ConnectWise 等のソフトに侵入。米 Microsoft が 10 月 21 日に報告

まとめ

ARTEX による被害は韓国の金融機関に留まらず、わずか数ヶ月で日本や世界中の数千組織へと一気に拡大しました。特定の標的だけでなく、「AI が自動で弱点を持つ組織を見つけ出して攻撃する」ため、業種や国境を問わず無差別に被害が連鎖しているのが特徴です。

例え話による解説

これはまるで、感染力の極めて高い新種のウイルスが、国境や海を越えてあっという間に世界的なパンデミックを引き起こしている状態です。特定の人が狙われているわけではなく、「免疫（セキュリティ対策）が少しでも弱い人（組織）」をウイルス（AI）が勝手に見つけ出して次々と感染させていくため、誰も対岸の火事ではいられません。

1.2.3 ARTEX の最新動向——拡散する脅威と対策の現状

- **タイムライン:** 2026 年初 GitHub 公開　2026 年 9 月韓国金融機関攻撃　10 月 8 日 CrowdStrike 特定　10 月 9 日 GitHub 公開停止　10 月 21 日 MS / Storm-0940 報告　現在～継続的拡散
- **拡散の実態:**
  - 独立配布版・カスタム版が増加（AhnLab）
  - 15 秒で脆弱性発見から情報抜き取り完了
  - 技術的ハードル低下、素人でも大規模攻撃が可能
  - ダークウェブや民間フォーラムでの流通拡大
- **各国・各社の対策状況:**
  - 韓国：緊急点検会議・消費者警報発令
  - 日本：GMO 等の警告、金融業界への注意喚起
  - MS Sentinel での検知ルール公開

  - IOC（侵害指標）公開（IP 38.244.50.120 等）
  - Zero Trust、Attack Surface 削減、PIM 導入
- **結論:** AI の民主化は攻撃の民主化でもある。従来の『高度な攻撃者だけが脅威』という前提は崩れた。

まとめ

AI 攻撃ツールの最も恐ろしい点は、コードが公開・拡散されたことで「高度な技術を持たない素人」でも簡単に大規模なサイバー攻撃が可能になった（攻撃の民主化）ことです。現在は AI を使って「15 秒」で情報窃取が完了するレベルにまで達しており、防御側は早急にゼロトラストなどの最新アーキテクチャへの移行を迫られています。

例え話による解説

例えるなら、「プロの殺し屋しか扱えなかった高性能なスナイパーライフル」が、ある日突然「ボタン一つで誰でも百発百中になる自動小銃」として闇市で大量に安売りされ始めたような状況です。これまでは「プロに狙われるような大企業」だけが警戒していればよかったのに、今や誰でも簡単に引き金を引けるため、町中のあちこちで素人による無差別な被害が起きているのです。

1.2.4 なぜ漏えいが続くのか——3 つの構造的欠陥

1. **入口の脆弱性:** VPN 脆弱性・未パッチの CVE、認証の弱さ（パスワード等）、委託先・サプライチェーン経由
2. **権限の問題:** 過剰な権限付与の常態化、セグメンテーションの不足、横展開の容易さ（ラテラルムーブメント）
3. **検知の遅れ:** 恒常的な監視の不足、異常検知の大幅な遅延、ログ管理・監査証跡の不備

対策: Just-in-Time Access（申請　承認　付与　作業　失効：時限的な権限付与により攻撃者が狙える時間を最小化）

まとめ

被害が後を絶たない背景には、大きく分けて「入口（弱い認証や未パッチ）」「内部の権限（誰でもどこでもアクセスできる状態）」「検知（侵入されても気づかない）」という 3 つの構造的な弱点があります。これを解決する有効な手段の一つが、必要な時だけ必要な権限を与える「Just-in-Time Access」です。

例え話による解説

これは、会社のセキュリティカードの管理に例えられます。「全社員が社長室から金庫まで、いつでもどこでも 24 時間入れる万能カード（過剰な権限）」を持たされている状態が現在の問題です。万が一そのカードが 1 枚でも盗まれたら終わりです。「Just-in-Time Access」とは、普段はどこにも入れないカードにしておき、「今から 1 時間だけ、この作業のために会議室 A の鍵を開けてください」と申請して、その時間・場所限定でしか使えない仕組みに変えることです。これなら盗まれても被害を最小限に抑えられます。

1.2.5 AI vs AI——攻撃者に対抗する AI 活用防御

攻撃側が AI を武器化するなら、防御側も AI で対抗する。

- **攻撃側の AI 利用:**
  - 脆弱性探索の自動化（ARTEX のような自動ペネトレーション）
  - 攻撃の高速化（43 秒で侵入など）
  - 大規模攻撃の自動化（Bissa Scanner による大量スキャン）
  - 人間の介入なしの連続攻撃
- **防御側の AI 活用:**
  - AI による異常検知（通常と異なる挙動のリアルタイム検出）
  - 脅威インテリジェンスの自動収集と分析
  - 自動対応・隔離（SOAR）
  - 行動分析（UEBA）
  - 予測的防御（攻撃パターンの予測とプロアクティブ対策）
- **AI 防御の効果:** 超高速な検知と対応、24 時間 365 日の監視、パターン学習で精度向上、誤検知の大幅な削減
- **キーメッセージ:** 速度には速度で、自動化には自動化で対抗する

まとめ

AI による超高速・自動化されたサイバー攻撃を人間の力だけで防ぐことは不可能です。「AI の攻撃には AI の防御で対抗する（AI vs AI）」のが現代のセキュリティの基本であり、防御側も AI を導入することで、24 時間体制でのリアルタイムな異常検知や、人間では追いつけないスピードでの自動隔離・対応が可能になります。

例え話による解説

例えるなら、音速で飛んでくる敵のミサイル（AI 攻撃）を、人間が目で見て大砲を撃って撃ち落とそうとしても絶対に間に合いません。迎撃するには、レーダーでミサイルを自動的に探知し、コンピューターが瞬時に軌道を計算して、自動で迎撃ミサイルを発射するシステム（AI 防御）が必要です。「機械のスピードには機械のスピードでしか勝てない」という、SF 映画のような防衛戦がすでに現実のネットワーク上で繰り広げられています。

1.3 ネットワーク防御要素 (PEP)

1.3.1 Switch (L2)

- **VLAN:** ネットワークの制御を行う技術（トラフィックを隔離する）
- **ACL:** （通常はインバウンドの）トラフィックを MAC アドレスや IP“‘latex

1.4　ネットワーク防御要素 (PEP)

1.4.1　スイッチ (L2) による防御

L2（データリンク層）のスイッチでは、主にネットワーク内部でのトラフィックの隔離と基本的なフィルタリングを行います。

- **VLAN:** ネットワークの制御を行う技術であり、トラフィックを論理的に隔離します。
- **ACL:**（通常はインバウンドの）トラフィックを MAC アドレスや IP アドレスに基づいてフィルタリングする技術です。
- **Bandwidth:** 各ポートごとの帯域（通信速度の上限）を設定し、トラフィックを制御します。

まとめ

L2 スイッチはネットワークインフラにおける「交通整理」の第一段階です。VLAN によってネットワークを分割して被害の拡散を防ぎ、ポート単位の ACL や帯域制限を組み合わせることで、物理的・論理的に近い領域でのセキュリティを確保します。

例え話による解説

例えるなら、オフィスビルの「フロア管理」です。VLAN は「部署ごとに部屋の壁を作って別々の空間にする」こと、ACL は「部屋の入り口にいる警備員が、社員証（MAC アドレス）を見て通してよいか決める」こと、そして Bandwidth（帯域）設定は「入り口のドアの大きさを変えて、一度に部屋に入れる人数を制限する」ような役割を果たしています。

1.4.2　ルータ (L3) によるトラフィック制御

L3（ネットワーク層）のルータ、特に AS 境界などの大きな接点では、パケットの破棄や迂回など、よりダイ “‘latex

1.5　ネットワーク防御要素 (PEP)

1.5.1　Switch (L2)

- **VLAN:** ネットワークの制御を行う技術（トラフィックを隔離する）
- **ACL:**（通常はインバウンドの）トラフィックを MAC アドレスや IP アドレスに基づいてフィルタリングする技術
- **Bandwidth:** 各ポートごとの帯域を設定

まとめ

L2 スイッチは主にデータリンク層で動作し、VLAN を用いたネットワークの論理的な分割（隔離）や、MAC アドレス等に基づく ACL によるアクセス制御、ポートごとの帯域制限を行うことで、ネットワーク内部の基本的な交通整理と防御を担います。

例え話による解説

例えるなら、巨大なオフィスビル（ネットワーク）の中で、「営業部」と「開発部」のフロアをしっかり壁で仕切って行き来できないようにし（VLAN）、各フロアへの入り口では「許可された社員証（MAC アドレス）」を持つ人しか通さない（ACL）ようにする、ビル内部の基本的な警備システムのようなものです。

1.5.2 ルータ (L3)

- **ACL:** 攻撃パケットを破棄（ドロップ）する
- **Policy-based Routing:** トラフィックを別のルートに迂回させる技術
- **Blackhole Routing:** AS 境界ルータにおいて不要なトラフィックを NULL デバイスに転送する技術
- **Sinkhole Routing:** AS 境界ルータにおいて不要なトラフィックを特定のポイントに転送する技術
- **Shunt Routing:** Sinkhole に送ったトラフィックを検査し、正常なトラフィックを再度戻す技術

まとめ

L3 ルータはネットワークの境界（IP レベル）でより高度な制御を行います。単にパケットを破棄するだけでなく、大規模な攻撃トラフィックを「行き止まり（Blackhole）」に捨てたり、一旦「隔離エリア（Sinkhole）」に誘導して検査した上で安全な通信だけを元のルートに戻す（Shunt）といった柔軟なトラフィック制御が可能です。

例え話による解説

例えるなら、高速道路の料金所や分岐点における警察の交通整理です。明らかに怪しい車はその場で通行止め（ACL ドロップ）にし、暴走族の大群が来たら、被害を出さないように「行き止まりの専用道路（Blackhole）」に誘導して消し去ります。あるいは、一旦「巨大な臨時駐車場（Sinkhole）」に全員を誘導して一台ずつ車内を調べ、無関係の一般車両だけを本線に戻す（Shunt）という高度な誘導作戦を行います。

1.5.3 NIDS における検知の仕組み

- **シグネチャ型:** 「悪意の核」の部分をデータベースに保持、パターンマッチ（例：EternalBlue の悪意のコード `\x4a\x6c\x4a\x6d\x49\x68\x43\x6c\x42\x73\x72\x0` と Snort による検知）
- **振る舞い検知型:** プロトコルのステート解析
- **ファイアウォール（境界型防御）の動作例:** 某京大学の FW ログ画面（通信先や通信内容によって危険度を色別に区別）

まとめ

NIDS（ネットワーク型侵入検知システム）は主に 2 つのアプローチで脅威を見つけ出します。一つは既知の攻撃コードの断片（シグネチャ）と通信内容を直接照合する「パターンマッチング」、もう一つは正常な通信の手順から逸脱した異常な挙動を捉える「振る舞い検知」です。これらを組み合わせ、FW 等で視覚的に危険度を分類して防御します。

例え話による解説

例えるなら、空港の優秀な保安検査官です。「シグネチャ型」は、あらかじめ渡されている「指名手配犯の顔写真リスト」と目の前の乗客を見比べて、一致したら即座に捕まえる仕組みです。一方「振る舞い検知型」は、顔はリストに載っていなくても、「真夏なのに分厚いコートを着て、異常に汗をかきながら周囲をキョロキョロしている」といった「不自然な行動」から危険を察知する仕組みです。この 2 つの目線を組み合わせることで、精度の高い警備を実現しています。

1.6　サイバー攻撃の手法 (1)：マルウェア

1.6.1　マルウェアの分類

- **ウイルス:** 他のプログラムに感染するプログラム
- **ワーム:** ネットワークを介して自己増殖するプログラム
- **トロイの木馬:** 正規のソフトを装った単体で動作する不正プログラム（他のプログラムに感染はしない）

まとめ

マルウェア（悪意のあるソフトウェア）は、大きく「ウイルス」「ワーム」「トロイの木馬」の 3 つに分類されます。他のファイルに寄生して広がるか（ウイルス）、単独でネットワーク上を自己増殖するか（ワーム）、有用なソフトを装って潜入するか（トロイの木馬）という活動様式の違いで区別されます。

例え話による解説

例えるなら、「ウイルス」は人間の風邪のように、人（ファイル）から人（ファイル）へと接触によってうつるものです。「ワーム」は自ら歩き回って勝手にどんどん増え続けるネズミや害虫のようなものです。そして「トロイの木馬」は、一見すると素敵な「プレゼント箱」なのに、家の中に持ち込んで開けた途端、中から泥棒が出てきて悪さをし始める罠のようなものです。

1.6.2　トロイの木馬の種別

目的や機能に応じて、以下のような多様な種類が存在します。

- **バックドア型:** 不正アクセスのための裏口を作る
- **ダウンローダー型:** 別の悪意あるプログラムを外部からダウンロードする
- **プロキシ型:** 踏み台として利用し、通信を中継する
- **キーロガー型:** キーボードの入力履歴を記録して盗む

- **パスワード窃盗型:** 保存されているパスワード情報を盗む
- **クリッカー型:** 勝手に広告などを不正クリックする
- **ボット型:** 外部から遠隔操作できる状態にする

まとめ

トロイの木馬は、侵入後の「役割」によって細かく分類されます。外部から操るための裏口（バックドア）を仕掛けるもの、キーボードの入力情報（パスワードなど）を盗み取るもの、他のマルウェアを呼び込むものなど、攻撃者の目的に合わせて様々な悪事を働きます。

例え話による解説

例えるなら、家に侵入した泥棒たちの「専門的な役割分担」です。勝手に家の裏口の鍵を開けておく係（バックドア）、外で待っている仲間を大量に家に呼び込む係（ダウンローダー）、金庫のダイヤルを回す指の動きを隠しカメラで監視する係（キーロガー）など、それぞれが特化した手口で被害を拡大させます。

1.6.3 トロイの木馬による被害事例 (1)

- **パソコン遠隔操作事件 (2012 年):** 他人の PC を遠隔操作し、ネット掲示板等で犯罪予告を行い、PC 所有者が誤認逮捕された事件。「便利なソフト」名目でバックドア機能を持つトロイの木馬をインストールさせ、掲示板を通じて命令を実行させました。
- **GameOver Zeus (2013 年):** オンラインバンキングを狙ったトロイの木馬。ばらまき型メールの添付ダウンローダーから感染し、Web ブラウザを監視して ID・パスワードやクレジットカード情報を窃取しました。

まとめ

トロイの木馬の恐ろしさを示す代表的な過去の事例です。「パソコン遠隔操作事件」では、サイバー攻撃が原因で無実の人が警察に誤認逮捕されるという社会問題に発展しました。また「GameOver Zeus」は、ネットバンキング利用者の金銭を直接狙う手口として猛威を振るいました。

例え話による解説

例えるなら、「遠隔操作事件」は、知らない間に自分の車が銀行強盗の逃走用に使われ、警察から「お前が犯人だ !」と疑われてしまうような恐ろしい状況です。「GameOver Zeus」は、ATM を操作している人の後ろにピタリと張り付き、手元の隠しカメラで暗証番号とカード情報を丸ごと盗み見ているスリのような犯罪が、ネット上で大規模に行われた状態です。

1.6.4 トロイの木馬による被害事例 (2)

- **Emdivi (2015 年 5 月):** 政府機関の端末が感染し 100 万件以上の個人情報が漏えい。リモートアクセス型トロイの木馬（RAT）であり、周到な標的型メール攻撃によって感染後、外部の C&C サーバーと通信を行いました。

– **Emotet (2019 年):** 大学職員の端末が感染し 1 万 8000 件以上のメール情報が流出。実在する企業や取引先を装った巧妙なメールの添付ファイルを開封させることで感染を広げました。

まとめ

近年では、特定の組織を狙い撃ちにする標的型攻撃（Emdivi）や、実在の人物のメールのやり取りを流用して本物そっくりの偽メールを送りつける攻撃（Emotet）など、人間の心理的な隙を突くソーシャルエンジニアリングの手法と組み合わされ、手口が極めて巧妙化しています。

例え話による解説

例えるなら、Emdivi は「凄腕の産業スパイ」が特定の省庁や大企業だけを狙い撃ちし、厳重な警備をすり抜けて機密書類をごっそり盗み出すようなものです。一方 Emotet は、上司や取引先からの「先日の見積書の件です。添付ファイルをご確認ください」という本物と見分けがつかない手紙を利用し、思わず封筒を開けた瞬間に毒ガスが広がるような、非常に狡猾で騙されやすい罠です。

1.6.5 その他のマルウェア形態

– **スマホにもスパイウェア:** Android の Google Play に登場したスパイアプリ（Privee Talk, TikTalk, MeetMe, Hello Chat など多数）。正規のメッセージアプリなどを装い、情報を抜き取ります。
– **ボットネット:** 司令塔となる C&C サーバーから、感染して操り人形となった PC 群（ボットネット/ゾンビ PC）へ命令を下し、一斉に不正クリックやサイバー攻撃を行わせます。
– **しつこいウィルス:** PC のハードディスクの最も根本的な部分である MBR (Master Boot Record) や Boot Sector に感染し、OS が起動する前に動作するため、駆除が非常に困難なタイプです。

まとめ

マルウェアの脅威はパソコンだけにとどまらず、スマートフォンの公式アプリストアに潜むスパイアプリや、何万台もの感染 PC を操って一斉攻撃を仕掛ける「ボットネット」、さらにはシステムの根本部分（MBR）に寄生して OS の初期化すら無効化する「しつこいマルウェア」など、攻撃の経路と手口は多岐にわたっています。

例え話による解説

例えるなら、スマホのスパイアプリは「便利なお小遣い帳アプリ」のふりをして実は盗聴器だった、という状況です。ボットネットは、催眠術（ウイルス）をかけられた何万もの人々（ゾンビ PC）が、親玉（C&C サーバー）の命令一つで特定のお店に一斉に押し寄せて営業妨害をするようなものです。MBR 感染は、家の壁紙にカビが生えたのではなく「家の基礎のコンクリート」に毒が練り込まれており、上モノを建て直してもすぐまた毒が染み出してくるような厄介さです。

1.7　ちょっと休憩：あなたの情報漏れてませんか？

1.7.1　漏洩確認サービス「';–have i been pwned?」の紹介

- **関谷の場合：** `sekiya@wide.ad.jp`　4 件のデータ漏洩で発見（Adobe 2013 年 1 億 5300 万件、Anti Public Combo List 2016 年 4 億 5800 万件、Lead Hunter 2020 年 6900 万件、Peatix 2019 年 420 万件）
- **大学のメールアドレス：** `sekiya@si.u-tokyo.ac.jp`　「Good news —no pwnage found!」

まとめ

「';–have i been pwned?」は、自分のメールアドレスや電話番号が、過去に発生した世界中のデータ漏洩事件で流出していないかを確認できる無料の Web サービスです。長年同じメールアドレスを使っている場合、自分でも忘れているようなサービスから、パスワードなどの情報とともに漏洩している可能性があります。

例え話による解説

例えるなら、自分の家の「合鍵」が過去にどこかの道端で落とされたままになっていないか、交番の「落とし物リスト」で検索するようなものです。もし自分の鍵（アドレス情報）がリストに載っていたら、泥棒に拾われる前にすぐに家の鍵穴（パスワード）を交換するなどの対策が必要です。

1.8　サイバー攻撃の手法 (2)：ネットワーク攻撃と認証

1.8.1　ネットワークを介した攻撃の事例

- Web アプリケーション脆弱性
- Remote Exploit
- DoS 攻撃
- ランサムウェア攻撃

まとめ

ネットワークを介した攻撃には、Web サイトのプログラムの弱点を突くものや、遠隔からシステムを乗っ取るもの、大量の通信を送りつけてサービスをダウンさせるものなど、様々な手法が存在します。これらはマルウェア感染とは異なり、ネットワーク越しに直接システムに危害を加えます。

例え話による解説

例えるなら、お店（Web サーバー）に対する外部からの嫌がらせです。お店の裏口の鍵が壊れているのを見つけて泥棒に入る（脆弱性・Remote Exploit）、無言電話を何万回もかけて通常の予約電話を受けられなくする（DoS 攻撃）、お店の金庫を勝手に別の鍵でロックして「開けてほしければ身代金を払え」と脅す（ランサムウェア）といった手口があります。

1.8.2　サイバーキルチェーンの 7 つの段階

1. 偵察 (RECONNAISSANCE)
2. 武器化 (WEAPONIZATION)
3. デリバリー / 配送 (DELIVERY)
4. エクスプロイト / 攻撃 (EXPLOITATION)
5. インストール (INSTALLATION)
6. 遠隔操作 (C&C: Command & Control)
7. 目的の実行 (ACTIONS ON OBJECTIVES)

まとめ

「サイバーキルチェーン」とは、軍事用語をサイバーセキュリティに転用したもので、攻撃者が標的を定めてから目的（情報の窃取やシステムの破壊など）を達成するまでの手順を 7 つの段階にモデル化したものです。防御側は、このプロセスのどこか一つでも攻撃の連鎖（チェーン）を断ち切る（キル）ことができれば、最終的な被害を防ぐことができます。

例え話による解説

例えるなら、映画に出てくる「綿密な銀行強盗の計画」です。まず銀行の警備状況を下見し（偵察）、専用の金庫破りツールを作り（武器化）、それをカバンに隠して持ち込み（配送）、警備員の隙を突いて中に入り（攻撃）、裏口を開けっ放しにする細工をして（インストール）、外の仲間とトランシーバーで連絡を取り合いながら（遠隔操作）、最終的にお金を奪って逃げる（目的の実行）という一連の流れです。

1.8.3　パスワード照合の仕組みとハッシュ化

- 利用者が入力したパスワードは、Web サーバ上でハッシュ関数によりハッシュ値に変換される
- 変換したハッシュ値とデータベースに保存されているハッシュ値を比較して認証する
- **ハッシュ化されたパスワードの解読:** CrackStation（ソルトなしハッシュの検索・解読サイト）
- **Slack のパスワード漏えい:** 2022 年 8 月、共有招待リンク機能のバグにより、ハッシュ化されたパスワードが約 0.5% のユーザーで漏洩していた事案
- **パスワードハッシュ生成とソルト:** [suspicious link removed] ではハッシュ値を一定にしないためのランダム要素「ソルト」が付与される）
- **総当たり攻撃ツールデモ:** Hydra を用いた RDP 認証へのブルートフォース攻撃映像

まとめ

システムはパスワードをそのまま（平文で）保存するのではなく、「ハッシュ関数」という一方向の計算を用いて暗号のような文字列（ハッシュ値）に変換して保存・照合します。さらに、同じパスワードでも異なるハッシュ値になるようランダムな文字列（ソルト）を付け加えることで、CrackStation のようなハッシュ辞書を用いた解読や、Hydra 等による総当たり攻撃（ブルートフォース）への耐性を高めています。

例え話による解説

例えるなら、パスワードが「生の豚肉」だとすると、ハッシュ化はそれを「ソーセージ」に加工するようなものです。一度ソーセージにすると元の豚肉の形に戻すこと（逆算）はできません。しかし、攻撃者は「豚肉ならこの味のソーセージになるはずだ」という対応リスト（辞書）を持っています。そこで、作る直前に「隠し味のスパイス（ソルト）」をランダムに混ぜることで、攻撃者のリストの味と一致しなくなり、元の肉が何だったのかをバレにくくする工夫がされています。

1.9 データベースの概要

1.9.1 フロントエンドとバックエンド

- **フロントエンド:** Web ブラウザ（HTML, JavaScript など、UI デザイン/開発）
- **バックエンド:** サーバシステム（Web/アプリケーションサーバ: Java, Ruby, Python など、DBMS: MySQL など、API/データ設計）
- **データベースの重要性:**
  - 通販サイト: 商品リスト、価格、在庫数
  - オンラインゲーム: ユーザ状況、ステータス、現在位置、アイテム状態
  - Amazon 等の通販やブログサイトの全挙動が DB で管理される
- **クラウドを用いた開発と運用 (Hexabase の例):** Web Browser (Request/JSON) API / Backend services / DB (IaaS: Azure/AWS/GCP) 各種外部サービス連携 (Salesforce, Google, AWS Lambda, Twilio, Slack 等)

まとめ

Web サービスは、ユーザーの目に触れる見た目や操作画面（フロントエンド）と、裏側でデータを処理する仕組み（バックエンド）に分かれています。中でもデータベース（DB）は、EC サイトの在庫やゲームのステータスなど、システムを動かすためのすべての情報を正確に記録・管理する心臓部としての役割を担っています。

例え話による解説

例えるなら、フロントエンドはお店の「客席やメニュー表」で、バックエンドは「厨房や裏の倉庫」です。そしてデータベースは、倉庫にある「在庫管理台帳」にあたります。お客さんが何を注文したか、倉庫にハンバーグがいくつ残っているかを完璧に記録しており、この台帳がないとお店は注文をさばけず、一瞬でパニックになってしまいます。

1.9.2 データベース管理システム (DBMS) とリレーショナルデータベース (RDBMS)

- **リレーショナルモデルとテーブル:**
  - 関係変数 (テーブル名)
  - 見出し: 属性（列 / Column / A1, A2, A3）
  - 本体: 組（行 / Row / タプル）
  - 値 (Value)

- **RDBMS の階層構造:** サーバプロセス　インスタンス　データベース　スキーマ　テーブル
- **一般的なデータベースシステムの構成要素:**
  - **インスタンス:** 共有メモリ（主にキャッシュ：データ書込み、ログ書込み、チェックポイント、システム監視等）、非共有メモリ、ユーザスレッド、バックグラウンドスレッド群
  - **(狭義/広義の) データベース:** トランザクションログファイル、アプリケーション用データファイル（テーブル/索引等）、システム用データファイル（メタデータ、ユーザ/権限/オブジェクト、統計情報、一時領域、UNDO）
- **SQL (Structured Query Language):**
  - リレーショナルモデルで表された集合を演算するための言語
  - 多くの RDBMS で採用され、自然言語に近く習得が容易
- **SQL が担う役割:**
  - データ定義言語 (DDL): データベースの構造を定義
  - データ操作言語 (DML): 格納されたデータを操作
  - データ制御言語 (DCL): アクセスを制御（ソフトウェアごとに実装差あり）

まとめ

RDBMS は、データを Excel のように「行」と「列」を持つ表（テーブル）の形式で管理するシステムです。システム内部は、高速に処理を行うための「メモリ（インスタンス）」と、データを永続的に保存する「ファイル（データベース）」で構成されています。これらのデータを定義・操作するための標準言語として「SQL」が使われます。

例え話による解説

これは「市役所の巨大な書類整理システム」のようなものです。RDBMS というシステムの中に「市民データベース」があり、さらにその中に「住民票テーブル（表）」が整理されています。そして SQL とは、窓口の職員に対して「○○町の 30 歳以上の人をリストアップして !」とお願いするための、世界共通の「専用の指示語（マニュアル言語）」と言えます。

1.9.3　SQL の具体例

- **テーブル例 : game_character**
  - 列: ID(id), 名前 (name), 職業 (job), レベル (level)
  - データ: (1, ヤマダ, 勇者, 30), (2, スズキ, 戦士, 42), (3, タナカ, 戦士, 25), (4, タカハシ, 魔法使い, 60), (5, ワタナベ, 僧侶, 20)
- **SELECT 文 (1):**
  - クエリ: `SELECT name, level FROM game_character WHERE level > 30`
  - 結果: スズキ (42), タカハシ (60)
- **SELECT 文 (2):**
  - クエリ: `SELECT * FROM game_character ORDER BY level desc;`
  - 結果: タカハシ (60), スズキ (42), ヤマダ (30), タナカ (25), ワタナベ (20) の順
- **UPDATE 文:**

* クエリ: `UPDATE game_character SET job="盗賊" WHERE name="タナカ";`

まとめ

SQL を使えば、膨大なデータの中から条件を指定して必要な情報だけを取り出したり（SELECT）、データを特定の順番に並べ替えたり（ORDER BY）、既存のデータを別の値に書き換えたり（UPDATE）することが、簡単な文法で実現できます。

例え話による解説

例えるなら、学校の全校集会で先生がマイクを使って指示を出しているようなものです。「レベル 30 以上の人だけ手を挙げて！（SELECT〜WHERE〜）」「レベルが高い順に 1 列に並んで！（ORDER BY〜）」「タナカ君は今日から職業を盗賊に変更して！（UPDATE〜）」というように、大量の生徒（データ）に対して一瞬で的確な操作を行うための魔法の呪文です。

1.9.4 データの検索と更新処理の内部動作

- **検索処理の動作:**
  1. ユーザが検索を実行。共有メモリ上に該当データがあるか確認
  2. 共有メモリに存在しない場合はディスク上のデータを検索
  3. 対象データ（ページ/ブロック単位）を共有メモリへコピー
  4. 検索結果をユーザに返す（他クライアントが同データを検索する場合はキャッシュから高速に返却）
- **更新処理の動作:**
  1. ユーザ A が 100 から 200 へ更新実行
  2. 変更履歴としてトランザクションログを生成、変更前の値 100 のスナップショットを生成
  3. ユーザが変更確定 (COMMIT) を実行
  4. トランザクションログに変更履歴が書き込まれる（完了するまで確定しない）
  5. スナップショットに廃棄フラグが立つ
  6. 次のチェックポイントのタイミングで共有メモリからデータファイルに更新データが書き込まれる

まとめ

データベースは処理を高速化しつつ、安全性を担保する工夫をしています。検索時は、ハードディスクを毎回読みに行くのではなく、一度読んだデータをメモリ（キャッシュ）に保持して高速化を図ります。更新時は、即座に重いデータファイルを書き換えるのではなく、まずは「変更の履歴（ログ）」だけを確実に記録して処理を完了（COMMIT）させ、後でまとめて本体に反映させます。

例え話による解説

検索処理は「図書館の司書さん」に似ています。よく聞かれる本は奥の書庫（ディスク）ではなく、すぐ手の届くカウンター（共有メモリ）に置いておき、次のお客さんにすぐ渡せるようにします。更新処理は「銀行の記帳」です。お金のやり取りがあったとき、いきなり大金庫（ディスク）のお金を動かすのではなく、まずは「取引記録のノート（トランザクションログ）」にしっかりメモを残して「手続き完了！」とします。その後、手の空いたタイミング（チェックポイント）で実際の金庫の中身を整理するのです。

1.10 SQL インジェクション

1.10.1 ユーザ認証の仕組みと脆弱性

- **ユーザ DB (Table):** (10, Alice, abcdefg), (20, Bob, 1234567), (30, Chris, password), (40, David, secret)
- **正常な認証処理:** `SELECT ユーザid FROM ユーザDB WHERE ユーザ名 = 'Alice' and パスワード = 'abcdefg'`
  レコードが存在するため認証成功
- **誤った認証処理:** `SELECT ユーザid FROM ユーザDB WHERE ユーザ名 = 'Alice' and パスワード = 'FOO'`
  該当レコードがなく認証失敗
- **SQL インジェクションの成立:** パスワード入力欄に `FOO' OR '1' = '1` を入力する。
  `SELECT ユーザid FROM ユーザDB WHERE ユーザ名 = 'Alice' and パスワード = 'FOO' OR '1' = '1'`
  条件文 `'1' = '1'` が常に真（True）となるため、正しいパスワードを入力したのと同じ扱いになり認証を突破される

まとめ

SQL インジェクションは、Web サイトの入力フォームなどに悪意のある SQL の断片（文字列）を入力し、データベースへの命令文を意図的に改ざんするサイバー攻撃です。パスワードを知らなくても、意図的に「常に正しい（True）」となる条件式を強制的に割り込ませることで、システムの認証を不正に突破されてしまいます。

例え話による解説

例えるなら、遊園地の入場ゲート（データベース）にいる警備員に対し、偽造した通行証を見せるようなものです。本来は「本物のチケットを持っていれば通す」というルールですが、攻撃者は「チケットを持っているか、または『1 は 1 である（当たり前の事実）』なら通す」と勝手にルールを書き換えて提示します。ルールにバカ正直な警備員は「チケットはないけど、1 は 1 だから通していいんだな」と勘違いして、泥棒を中に入れてしまうのです。

1.10.2 インジェクション攻撃防御の難しさ (スライド 94)

- **システム的な難しさ:** Web システム、DB システム、Web プログラミング言語のそれぞれが単体で安全であっても、組み合わせることで問題が発生してしまう（「設計上の欠点」）。
- **当時特有の問題:**

* Web システムのニーズが非常に高かった時代背景
* Web システムを作成するためのフレームワーク（現在の Ruby on Rails に相当するもの等）がまだ使われていなかった
* 入力に対する出力の検査（エスケープ処理や無害化など）が必要であるという認識自体が乏しかった

まとめ

SQL インジェクションが過去に猛威を振るった背景には、「Web サーバ・プログラミング言語・データベース」がそれぞれ単体では正常に動くため、組み合わせた時の脆弱性に誰も気づきにくかったという構造的な問題があります。また、当時はセキュリティを担保する「フレームワーク」が普及しておらず、開発者自身に「入力された文字を無害化する（エスケープ処理）」というセキュリティ意識が不足していました。

例え話による解説

例えるなら、「頑丈なレンガ」「優秀な接着剤」「腕の良い職人」がそれぞれ単体としては完璧でも、「レンガの間に接着剤を塗る」という共通ルールが共有されていなかったため、壁に隙間ができてしまったような状態です。当時は「あらかじめ隙間ができないように作られた便利な家の組み立てキット（フレームワーク）」が存在せず、みんなが手探りで急いで家（Web システム）を建てていたため、泥棒が侵入できる隙間があちこちに生まれてしまいました。